

Step-by-Step Implementation: Migrating to Azure ML Compute Cluster

This document provides a technical walkthrough for transitioning your `house-price` project from a Compute Instance to a Compute Cluster. This transition specifically resolves the current blocker where **disabled root access** on Compute Instances prevents data scientists from submitting jobs.

1. The Core Problem: Root Access & Job Submission

In your current setup, root access is disabled on **Compute Instances** for security. This creates a critical failure point:

- **The Error:** When a data scientist tries to run a pipeline or submit a job, Azure ML often attempts to build a local Docker environment or interact with the Docker daemon.
 - **The Blocker:** Without root access, the user cannot trigger these Docker operations. Consequently, the job fails to even "start" because the environment cannot be prepared on the restricted instance.
 - **The Result:** Data scientists are stuck and cannot run their "Learning Tasks" or pipelines in the cloud.
-

2. The Solution: Moving to Compute Cluster

By moving to a **Compute Cluster**, we change *where* the privileged operations happen:

- **Cloud-Side Build:** Instead of building the environment on the user's restricted machine, Azure ML builds it in the cloud (using Azure Container Registry). This process does not require the user to have root access.
 - **Cluster Permissions:** As per the plan, the **Compute Cluster** will have the necessary permissions (root access for the backend) to pull images from the Microsoft Container Registry (MCR) and execute the code.
 - **User Security:** Data scientists stay on their restricted, secure workstations but offload the "privileged" execution to the managed cluster.
-

3. Cost Analysis: Standard_F8s_v2

You are using the **Standard_F8s_v2** (8 Cores, 16 GB RAM). Below is the cost breakdown for your team of 20 data scientists.

Metric	20 Compute Instances (Always On)	1 Compute Cluster (Autoscale 0-20)
Hourly Rate (Est.)	~\$0.34 per node ¹	~\$0.34 per active node
Daily Cost (8h)	~\$54.40 (Fixed)	~\$54.40 (Only if all 20 run for 8h)
Idle Cost (Night)	~\$108.80 (16h)	\$0.00
Monthly Total	~\$4,800+	~\$1,200 - \$1,600 (Est. actual usage)

4. UI-Based Implementation (Step-by-Step)

Step A: Create the Cluster (Studio UI)

1. Open [Azure ML Studio](#).
2. Go to **Manage** -> **Compute** -> **Compute clusters** -> **+ New**.
3. **Select VM Size:** Search for `Standard_F8s_v2` . Click **Next**.
4. **Settings:**
 - **Compute name:** `shared-training-cluster`
 - **Min nodes:** `0` (Critical for saving money)
 - **Max nodes:** `20`
 - **Idle seconds before scale down:** `120`
5. **Advanced Settings:** Ensure **Managed Identity** is enabled. This identity will have the root-level permissions needed to pull from MCR.
6. Click **Create**.

Step B: Create the Environment (Studio UI)

1. Go to **Assets** -> **Environments** -> **Custom environments** -> **+ Create**.
2. **Name:** `kedro-house-price-env` .
3. **Select source:** "Upload existing directory" (Upload your `house-price` folder).
4. **Environment type:** Select "Conda with Docker image".

5. **Docker settings:** Use the default MCR image: `mcr.microsoft.com/azureml/openmpi4.1.0-ubuntu20.04` .
 6. Click **Next** and **Create**. Azure ML handles the build in the cloud, bypassing the user's local root restriction.
-

5. Code Implementation: The Submission Fix

To run the pipeline from a restricted notebook, you must use the Python SDK to "submit" the job to the cluster. This avoids local Docker builds.

Create `submit_job.py`

Python

```
from azure.ai.ml import MLClient, command
from azure.identity import DefaultAzureCredential

# 1. Connect to your workspace
ml_client = MLClient.from_config(credential=DefaultAzureCredential())

# 2. Define the job (This sends the request to the cloud, no local root needed)
job = command(
    code=".",
    command="python -m kedro run",
    environment="kedro-house-price-env@latest",
    compute="shared-training-cluster",
    experiment_name="house_price_migration"
)

# 3. Submit
returned_job = ml_client.jobs.create_or_update(job)
print(f"Job submitted successfully! Monitor here: {returned_job.studio_url}")
```

6. Summary for Management

1. **Solves the Blocker:** By offloading the "Job" to the cluster, we bypass the root access restriction on user instances that is currently causing submission failures.
2. **Maintains Security:** Data scientists remain without root access on their workstations, while the Cluster handles the necessary privileged operations in a controlled environment.

3. **Reduces Cost:** The autoscale feature ensures we only pay for the `Standard_F8s_v2` nodes when a job is actually running.

References

[1] Azure VM Pricing (Fsv2-series):

[2] Azure ML Compute Cluster Autoscale: